Chapter 8

Value Functions: From Scores to Decisions

8.1 Beyond Relevance: Why Scores Are Not Enough

The multi-task ranking model from Chapter 7 produces a set of predicted probabilities for each candidate song in the Yandex dataset: P(listen), P(like|listen), E[play_ratio], P(dislike|listen). But these raw scores do not answer the question that ultimately matters: which items should appear at the top of the user's feed?

A track with P(listen) = 0.9 and P(dislike|listen) = 0.4 is likely to be started but also likely to annoy the user. A track with P(listen) = 0.5 and P(like|listen) = 0.6 is less certain to be played but, conditional on being played, has a high chance of delighting the user. Which should rank higher? The answer depends on what the business values.

The value function is the mechanism that encodes these business priorities. It takes the multi-task predictions as input and produces a single composite score that determines the final ordering. In Instagram Explore's architecture, this is called the 'value model': a weighted combination of predicted engagement probabilities that approximates the value each piece of media brings to the user.

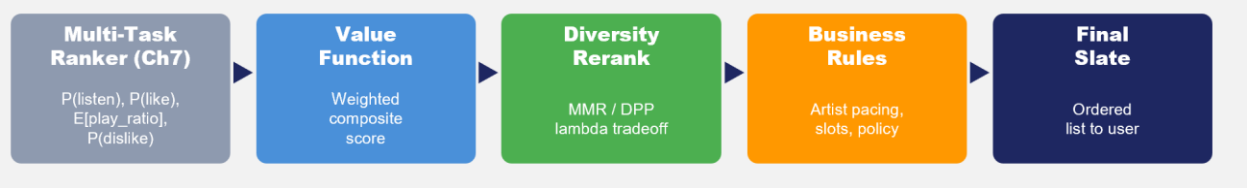


Figure 8.1: Pipeline showing Multi-Task Model outputs (P(listen), P(like), E[play_ratio], P(dislike)) -> Value Function (weighted combination) -> Value Score -> Diversity Reranking -> Business Rules -> Final Ranked List

*Author's note: The value function stage does not necessarily improve relevance metrics and may in fact be detrimental to the more conventional narrower accuracy metrics. Consider a user who knows exactly what they want when shopping: showing them diverse, serendipitous items **could** frustrate them. Business team should frame this as tradeoffs: 'We accept an X% decrease in short-term CTR for a Y% increase in long-term retention.' Without this alignment across the org, introducing a value function module after the re-ranking step turns into an optimization problem without a clear objective.*

8.2 The Value Function Formula

The value function combines multi-task predictions into a single score using a weighted linear combination with conditional composition:

$$Value = w_listen * P(listen) + w_like * P(listen) * P(like|listen) + w_engagement * P(listen) * E[play_ratio] - w_dislike * P(listen) * P(dislike|listen)$$

The Pythonic implementation is straightforward:

```
# In code (from value_function.py):
value = (w_listen * p_listen
       + w_like * p_listen * p_like
       + w_engagement * p_listen * e_engagement
       - w_dislike * p_listen * p_dislike)
```

8.2.1 Conditional Composition

Notice that the *like, engagement,* and *dislike* terms are multiplied by P(*listen*). This conditional composition is a deliberate design choice: downstream signals should only contribute to the value score when there is a reasonable probability of initial engagement. A track with P(listen) = 0.01 and P(like|listen) = 0.9 should not receive a high value score from the like term, because the user is almost certainly not going to hear it in the first place.

This mirrors real-world conditional probabilities. The joint probability of a user listening and liking a track is P(listen) * P(like|listen), not P(like|listen) alone. The value function uses this joint probability structure to ensure that all terms are grounded in realistic engagement scenarios.

8.2.2 Weight Selection

The weights (w_listen, w_like, w_engagement, w_dislike) encode the relative importance the business places on each outcome. These are often not learned parameters and instead are set by the product or business team based on strategic priorities.

Strategy	w_listen	w_like	w_dislike
Engagement-focused	1.0	1.0	-1.0
Satisfaction-focused	0.5	10.0	-20.0
Growth-focused	1.0	5.0	-10.0

An engagement-focused strategy prioritizes raw listens, suitable for a platform trying to maximize time spent. A satisfaction-focused strategy heavily weights likes and penalizes dislikes, suitable for a platform optimizing for long-term retention. The growth-focused strategy balances both.

Author's note: In practice, value function weights are tuned through online A/B tests, not exclusively through offline optimization. You hypothesize a set of weights, deploy them to a small percentage of traffic, measure business KPIs (retention, revenue, session length), and iterate. The offline tutorial demonstrates how different weights change the ranking order, but the final calibration happens in production.

8.2.3 Experimenting with Weight Configurations

The code tutorial provides a lightweight framework for exploring weight configurations on pre-computed predictions from Chapter 7. By loading the validation predictions Parquet file (the test predictions file is kept for reporting the chosen weights) and applying different weight vectors, you can observe how the ranking changes without re-running model inference:

- Equal weights produce a baseline ordering that reflects the model's raw predictions
- Increasing w_like surfaces items the model predicts will receive explicit positive feedback
- A large negative w_dislike acts as a safety mechanism, demoting items likely to cause negative reactions even if they have moderate engagement probability
- The interplay between weights creates non-obvious effects: a high w_like with a high w_dislike penalty may demote polarizing content that would score well on either signal alone



Figure 8.2: Three strategies for tuning value-function weights, arranged along a maturity axis. Manual tuning (left) is fast and interpretable but does not adapt to drift — the right starting point for small teams and MVPs. A/B grid sweeps (middle) empirically validate weight configurations against real users and are the default for mature teams with meaningful traffic. Bandits or reinforcement-learning approaches (right) continuously adapt weights and can personalize them per context, at the cost of higher engineering

complexity and careful reward design. The progression tracks team maturity, traffic volume, and personalization needs.

8.3 Beyond-Accuracy Metrics

Pure relevance optimization leads to predictable, homogeneous recommendation lists. A user who listened to three jazz tracks should not receive a feed of 20 jazz tracks. Beyond-accuracy metrics quantify the qualities of recommendation lists that relevance alone cannot capture.

8.3.1 Diversity

Intra-List Diversity (ILD) measures how different the items within a single recommendation list are from each other. It is typically computed as the average pairwise distance between item embeddings in the list:

*ILD = (2 / (k * (k-1))) * sum over all pairs (i,j) of distance(item_i,item_j)*

where the distance function is often 1 - cosine_similarity using the item embeddings from Chapter 10.

```
# Intra-List Diversity (from evaluate.py):
from sklearn.metrics.pairwise import cosine_similarity

def compute_ild(item_ids, embeddings, top_k=10):
    vecs = [embeddings[i] for i in item_ids[:top_k]
            if i in embeddings]
    sim = cosine_similarity(np.array(vecs))
    k = len(vecs)
    ild = 1.0 - (sim.sum() - k) / (k * (k - 1))
    return ild
```

Catalog Coverage (or aggregate diversity) measures what fraction of the total item catalog is recommended across all users. A system that only recommends the top 100 most popular items has low coverage, even if each individual list looks diverse.

8.3.2 Novelty

Novelty captures whether the system recommends items the user has not seen before and would not have discovered on their own. It is often operationalized as the average inverse popularity of recommended items:

Novelty = average over items in list of -log2(P(item))

where P(item) is the item's popularity (fraction of users who have interacted with it). Recommending long-tail items yields higher novelty scores.

8.3.3 Serendipity

Serendipity combines novelty with relevance: an item is serendipitous if it is both unexpected and relevant. A random item from the catalog is novel but not serendipitous

if the user would not enjoy it. Serendipity is harder to measure quantitatively and is often assessed through user studies or proxy metrics.

8.3.4 System-Level Metrics

- **Gini coefficient:** Measures inequality in how often items are recommended. A Gini of 0 means all items are recommended equally; a Gini near 1 means a few items dominate.
- **Shannon entropy:** Quantifies the information content of the recommendation distribution. Higher entropy means more uniform coverage.
- **Effective Catalog Size (ECS):** Popularized by Netflix, this measures the 'effective' number of items receiving meaningful exposure, accounting for the long-tail distribution of recommendations.

8.4 Diversity Reranking with MMR

After the value function produces a relevance-ordered list, the diversity reranking stage adjusts the ordering to increase variety while preserving most of the relevance. The most widely used approach is Maximal Marginal Relevance (MMR).

8.4.1 The MMR Algorithm

MMR (Carbonell & Goldstein, 1998) iteratively selects items that balance relevance to the user with dissimilarity to items already selected:

*MMR(i) = lambda * Relevance(i) - (1 - lambda) * max_{j in S} Similarity(i, j)*

where S is the set of already-selected items and lambda controls the relevance-diversity tradeoff:

- lambda = 1.0: Pure relevance (no diversity), equivalent to the value function ranking
- lambda = 0.0: Pure diversity (ignores relevance), selects the most different items regardless of quality
- lambda = 0.7: A common starting point that preserves most relevance while introducing meaningful diversity

```
# MMR greedy selection (from mmr_diversity.py):
selected = []
for _ in range(top_k):
    best_item, best_score = None, -np.inf
    for item_id in remaining:
        relevance = score_map[item_id]
        if not selected:
            penalty = 0.0
        else:
            penalty = max(cosine_sim(emb[item_id], emb[s])
                          for s in selected)
```

```
        mmr = lam * relevance - (1 - lam) * penalty
        if mmr > best_score:
            best_score, best_item = mmr, item_id
    selected.append(best_item)
    remaining.remove(best_item)
```

8.4.2 Similarity Function

The similarity function in MMR determines what 'diversity' means. In the code tutorial, we use cosine similarity between item embeddings (audio embeddings from the Yambda dataset). This captures acoustic similarity: MMR will avoid placing multiple acoustically similar tracks adjacent to each other.

Alternative similarity functions capture different notions of diversity:

- **Content-based:** Genre, artist, album metadata. Ensures variety in content categories.
- **Embedding-based:** Learned item embeddings from Chapter 10. Captures latent similarity patterns.
- **Hybrid:** Weighted combination of content and embedding similarity.

8.4.3 Computational Considerations

MMR is a greedy algorithm with O(k * n) complexity where k is the number of items to select and n is the candidate pool size. Since k is typically 10-30 and n is typically 50-200 at this stage, the computation is lightweight. However, pre-computing the similarity matrix can be expensive for large embedding dimensions; caching and approximate methods help.

8.5 Business Rules and Pacing

The final layer of the ordering pipeline applies deterministic business constraints that cannot be easily encoded in a learned model. These rules reflect hard product requirements rather than soft optimization objectives.

8.5.1 Artist Pacing

A recommendation list should not contain too many tracks from the same artist, even if the model predicts high engagement for all of them. Artist pacing limits the maximum number of items per artist in the final list:

- If max_per_artist = 2, the third track by the same artist is demoted regardless of its value score
- This prevents the 'artist takeover' phenomenon where a popular artist dominates the feed
- Similar pacing can be applied at the album level or genre level

```
# Artist pacing (from business_rules.py):
artist_counts = {}
paced, deferred = [], []
for item_id in ranked_items:
    artist = artist_map.get(item_id)
    if artist is None or artist_counts.get(artist, 0) < max_per_artist:
        paced.append(item_id)
        if artist:
            artist_counts[artist] = artist_counts.get(artist, 0) + 1
    else:
        deferred.append(item_id)  # push to end
paced.extend(deferred)
```

8.5.2 Slot Allocation

Some positions in the recommendation list may be reserved for specific purposes:

- Discovery slots: positions reserved for items from outside the user's established preferences
- Promotion slots: positions allocated for editorial picks, new releases, or sponsored content (we will cover ads in the next chapter)
- Freshness slots: positions that must contain items not previously shown to the user

8.5.3 Policy Constraints

Hard constraints that override all other signals:

- Content policies: items flagged for quality or safety issues are removed regardless of score
- Geographic restrictions: items unavailable in the user's region are filtered out
- User blocks: items from creators the user has blocked are always excluded
- Rate limiting: preventing the same item from appearing in consecutive sessions

Author's note: Some Business rules can only be applied as the final step, after the value function and diversity reranking. Applying them earlier can distort the optimization: if you remove items before scoring, the model cannot learn from the absence.

8.6 Alternative Approaches to Reranking

While MMR is the most common heuristic approach, the reranking literature offers several alternatives for teams with more complex diversity requirements.

8.6.1 Determinantal Point Processes (DPPs)

DPPs provide a probabilistic framework for selecting diverse subsets. They define a probability distribution over subsets of items where the probability of selecting a subset

is proportional to the determinant of a kernel matrix over those items. Subsets with diverse items have higher determinants and thus higher selection probability.

- More principled than MMR: jointly optimizes the entire selected set rather than greedily one item at a time
- Computationally heavier: exact DPP sampling is O(n^3) in the candidate set size
- Pinterest has used DPPs for diversifying recommendations, evolving from simpler round-robin approaches

8.6.2 Submodular Optimization

Diversity objectives are often submodular: the marginal gain of adding a new diverse item decreases as the set grows. Submodular optimization provides greedy algorithms with provable (1 - 1/e) approximation guarantees. This theoretical grounding makes it attractive for teams that want formal diversity guarantees.

8.6.3 Learned Rerankers

Recent work has explored end-to-end learned reranking models that jointly optimize relevance and diversity:

- **Listwise Transformers:** Apply self-attention over the candidate set to model inter-item dependencies and directly output a diverse, relevant ordering.
- **Reinforcement Learning:** Treat the sequential item selection as a Markov Decision Process and learn a policy that maximizes long-term reward (including diversity).
- **LinkedIn's LiGR:** A transformer-based reranker that models item interactions within the list to optimize for both engagement and diversity in feed ranking.

For most teams starting out, a manual set of business rules or MMR provides the best complexity-to-impact ratio. Move to DPPs or learned rerankers only when you have strong evidence that MMR's greedy approach leaves measurable diversity gains on the table.

8.7 Industry Perspectives

8.7.1 Instagram Explore

Instagram Explore's value model combines predicted engagement probabilities with manually tuned weights:

*Value = W_click * P(click) + W_like * P(like) - W_see_less * P(see less)*.

The weights are set based on the product team's priorities and adjusted through online experiments. This is precisely the pattern our code tutorial implements.

8.7.2 LinkedIn Feed

LinkedIn evolved from rule-based session-level diversity (limiting posts from the same author or topic within a feed) to LiGR, a transformer-based reranker that models inter-item effects. Their key finding was that simple rules left significant value on the table: a learned model could achieve better diversity-relevance tradeoffs by understanding which combinations of posts work well together.

8.7.3 Netflix

Netflix popularized the Effective Catalog Size (ECS) metric to measure how much of their catalog gets meaningful exposure. They found that purely relevance-optimized recommendations led to a small fraction of titles receiving most views, creating a 'popularity trap.' Their reranking strategies aim to expand ECS while maintaining member satisfaction.

8.7.4 Pinterest

Pinterest evolved their diversity approach from round-robin (alternating content types) to DPPs (probabilistic diverse subset selection). They specifically addressed representation diversity, ensuring that recommendations for queries like 'wedding hairstyles' included diverse skin tones and hair types, not just the most popular images.

8.8 End-to-End Pipeline Design

The complete ordering pipeline is a sequence of composable stages, each with a clear responsibility

8.8.1 Modularity

Each stage in the pipeline is designed to be independently tunable and testable. The value function weights can be adjusted without retraining the ranking model. The MMR lambda can be tuned without changing the value function. Business rules can be updated instantly without any model involvement.

This modularity is a key advantage of the multi-stage approach over attempting to bake all objectives into a single model. A monolithic model that jointly optimizes relevance, diversity, and business constraints would be harder to train, harder to debug, and harder to evolve as business priorities change.

8.8.2 Latency Budgeting

The ordering pipeline adds latency on top of the ranking model:

Stage	Typical Latency	Bottleneck
Value function	< 1ms	Simple arithmetic
MMR reranking	1-5ms	Similarity lookups

Business rules	< 1ms	Rule evaluation
Total ordering	2-7ms	Dominated by MMR

The ordering pipeline is lightweight compared to the ranking model itself (100-200ms). Pre-computing and caching the item similarity matrix for MMR is the main optimization lever.

8.8.3 Fail-Safes

The ordering pipeline should degrade gracefully:

- If the MMR similarity lookup fails, fall back to the value function ordering (relevance without diversity)
- If the value function receives invalid predictions, fall back to the primary task's raw score (e.g., P(listen))
- If business rule data is stale, apply conservative defaults (e.g., stricter pacing limits)
- Log all fallback events for monitoring and alerting

8.9 Code Tutorial: Value Functions and Diversity on Yambda

The code tutorial for this chapter implements the complete ordering pipeline on top of the MMoE predictions from Chapter 7.

8.9.1 Value Function Module

The ValueFunction class takes configurable weights and computes the composite score using conditional composition. The tutorial demonstrates how different weight configurations change the ranking order and how to evaluate each configuration using both relevance (NDCG) and diversity (ILD, artist entropy) metrics.

```
from chapter8_value_functions import ValueFunction, ValueFunctionConfig

config = ValueFunctionConfig(
    w_listen=1.0, w_like=2.0,
    w_engagement=1.5, w_dislike_penalty=1.0
)
vf = ValueFunction(config)

# Compute composite scores from multi-task predictions
values = vf.compute_value(
    p_listen=predictions["p_listen"],
    p_like=predictions["p_like"],
    e_engagement=predictions["e_engagement"],
    p_dislike=predictions["p_dislike"]
)
```

```
# Rank candidates by value score
ranked_indices = np.argsort(-values)
```

8.9.2 MMR Diversity Module

The MMRReranker uses the Yambda audio embeddings to compute item similarity. The tutorial sweeps lambda from 0.5 to 1.0 and visualizes the relevance-diversity tradeoff curve, showing the Pareto frontier of achievable NDCG-ILD combinations.

```
from chapter8_value_functions import MMRReranker, DiversityConfig

reranker = MMRReranker(
    embeddings=item_embeddings,  # from Yambda audio features
    config=DiversityConfig(lambda_param=0.7, top_k=20)
)

# Rerank value-scored candidates for diversity
candidates = [(item_id, score)
              for item_id, score in zip(item_ids, values)]
diverse_ranking = reranker.rerank(candidates)
```

8.9.3 Business Rules Module

The ArtistPacer implements artist-level pacing using the Yambda artist-item mapping. The tutorial shows how different max_per_artist thresholds affect artist entropy and overall list quality.

```
from chapter8_value_functions import ArtistPacer, PacingConfig

pacer = ArtistPacer(
    artist_mapping=artist_map,  # item_id -> artist_id
    config=PacingConfig(max_per_artist=2, max_per_album=1)
)

paced_list = pacer.apply_pacing(diverse_ranking)
```

8.9.4 End-to-End Pipeline

The OrderingPipeline composes all three stages. To evaluate it end to end, run the full pipeline on the test set predictions, with weights fixed in advance (any tuning uses the validation predictions), and report:

- NDCG@K at each pipeline stage (to quantify relevance cost of each transformation)
- ILD at each stage (to quantify diversity improvement)
- Artist entropy at each stage (to measure content source diversity)
- Catalog coverage across all users (to measure system-level health)

```
from chapter8_value_functions import (
    OrderingPipeline, PipelineConfig,
```

```
    ValueFunctionConfig, DiversityConfig, PacingConfig
)

pipeline = OrderingPipeline(PipelineConfig(
    value_config=ValueFunctionConfig(
        w_listen=1.0, w_like=2.0, w_dislike_penalty=1.0
    ),
    diversity_config=DiversityConfig(lambda_param=0.7),
    pacing_config=PacingConfig(max_per_artist=2)
))

# Full pipeline: value function -> MMR -> business rules
final_ranking = pipeline.rerank(
    candidate_ids=candidate_ids,
    predictions=multitask_predictions,
    top_k=20
)
```

8.9.5 Key Takeaways from the Tutorial

- The value function allows the same model to serve different business strategies by changing weights alone
- Conditional composition (multiplying by P(listen)) is critical: without it, the dislike penalty can distort rankings by penalizing items the user would never encounter anyway
- MMR at lambda = 0.7 provides meaningful diversity improvement with modest relevance cost
- Artist pacing has a larger effect on perceived diversity than embedding-based MMR, because users perceive artist repetition more strongly than acoustic similarity
- The full pipeline (value function + MMR + pacing) yields a substantially different ranking than raw model scores, demonstrating that the ordering stage is not merely cosmetic

8.10 Chapter Summary

- The value function translates multi-task model predictions into a single composite score that encodes business priorities
- Conditional composition (multiplying downstream signals by P(listen)) ensures realistic probability grounding
- Value function weights are business decisions, not model parameters: they are set by product teams and validated through A/B tests
- Beyond-accuracy metrics (ILD, novelty, serendipity, coverage) quantify list quality dimensions that relevance alone cannot capture

- MMR provides a principled, efficient approach to diversity reranking with a single tunable parameter (lambda)
- Business rules (pacing, slot allocation, policy constraints) apply hard constraints as the final pipeline stage
- The ordering pipeline is modular: each stage is independently tunable, testable, and replaceable
- Alternative approaches (DPPs, learned rerankers) offer stronger theoretical guarantees but at higher complexity; start with MMR and upgrade when evidence justifies it

References

- Carbonell, J. & Goldstein, J. (1998). *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries.* SIGIR 1998.
- Meta Engineering. (2023). *Scaling Instagram Explore Recommendations.* engineering.fb.com.
- Yan, E. (2022). *System Design for Recommendations and Search.* eugeneyan.com.
- Kulesza, A. & Taskar, B. (2012). *Determinantal Point Processes for Machine Learning.* Foundations and Trends in Machine Learning.
- LinkedIn Engineering. (2023). *LiGR: Improving Feed Ranking with Learned Inter-item Dependencies.* engineering.linkedin.com.
- Netflix Technology Blog. (2022). *Effective Catalog Size and Content Discovery.* netflixtechblog.com.
- Pinterest Engineering. (2022). *Representation-Aware Recommendations.* medium.com/pinterest-engineering.
- Ma, J. et al. (2018). *Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts.* KDD 2018.
- Shaped.ai. (2023). *The Anatomy of Modern Ranking Architectures: Part 4.* shaped.ai/blog.
- Yandex. (2025). *Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking and Retrieval.* arXiv:2505.22238.